

formHTR: Template-Guided OCR Extraction of Handwritten Content from Scanned Forms

Matej Troják¹, Stéphane Pesant², Jan Glos³, Kerstin Leberecht¹, Michael Kuhn¹, and Peer Bork¹

¹ EMBL, Heidelberg, Germany ² EMBL-EBI, Hinxton, UK ³ Masaryk university, Czech republic ¶ Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [✉](#)

Submitted: 11 June 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

formHTR is a Python software package for automatic handwritten text recognition (HTR) and extraction from scanned form documents. While the content extraction itself is accomplished using advanced pretrained OCR models, a key step in the process is considering the prior knowledge of the expected contents. That is achieved by the precise, semi-automatic annotation of the regions of interest (ROIs) in the form template, specifying the locations and content types of ROIs. Using this approach in combination with comparing and evaluating outputs from multiple OCR models significantly improves the quality of the extraction.

Statement of need

Large-scale scientific expeditions often collect huge amounts of samples, with a need to record the origin and context of the samples and observed features (metadata). While digital metadata collection methods are becoming more popular, paper forms (so called *logsheets*) are still the most popular method for their reliability in extreme environments, stability, natural scalability, and ease to use (Brewer & Guiterman, 2016; van Tamelen, 2004).

It is necessary to build an infrastructure that processes logsheets automatically with a minimal amount of manual interventions and laborious proofreading. Optical character recognition (OCR) methods (Mori et al., 1992) are used to extract the content from a scanned logsheet, with additional complexity added by handwritten type of content. While training custom models on a particular handwriting style of a person is generally more precise, this approach is impractical for large-scale expeditions with high turnaround of staff and consequently higher amount of distinct handwritings. The use of multiple pretrained general purpose OCR models, allowing consensus or majority decision making, is a more suitable approach.

Additionally, assuming a large-scale expedition enforces certain standards on the sample collection process, so does it on the metadata level. That means standardised logsheets are often developed, and used repeatedly in various sampling scenarios. As a consequence, to digitalise the contents of such logsheets, we can leverage their known structure and expected content types to navigate the OCR methods for more reliable and precise results.

State of the field

The current OCR landscape of pretrained tools can be split between open-source models and cloud-based services. On the open-source side, there are many tools such as Tesseract OCR (Google & HP, 2021), EasyOCR (Jaided AI, 2024), OCR4All (Reul et al., 2019), and PaddleOCR (Cui et al., 2025) which provide pretrained models or optionally allow to train

39 custom models. Some of the trade-offs are in accuracy, speed, and deployment complexity.
40 Cloud-based pretrained OCR services, including Google Cloud Vision API (Google LLC, 2024),
41 Azure AI Document Intelligence (Microsoft, 2024), and Amazon Textract (Amazon Web
42 Services, 2024), are accessible via APIs and are highly optimised for structured documents and
43 provide higher-level outputs (e.g. key-value pairs).

44 Naturally, the combination of multiple tools, models, or services has emerged. Systems such
45 as OCRmyPDF (Barlow, 2026) or unified interfaces like OcrPy (maxentlabs, 2022) integrate
46 engines like Tesseract, cloud APIs, and downstream processing into a single pipeline, effectively
47 abstracting over multiple OCR backends. The tool Handprint (Caltech Library, 2020) combines
48 multiple cloud services and outputs annotated images or raw results, as well as compares the
49 recognized text to some level of the ground truth (expected content)¹.

50 Software design

51 formHTR is structured into two main steps with an overview in Figure 1. The specification
52 step selects regions of interest (ROIs) from a template and assign them a meaning. This is
53 achieved by selecting and manipulating the region locations, while the variable names and
54 ROI types are assigned to individual ROIs. The output of the specification step is a config file
55 containing position, name, and type of identified ROIs.

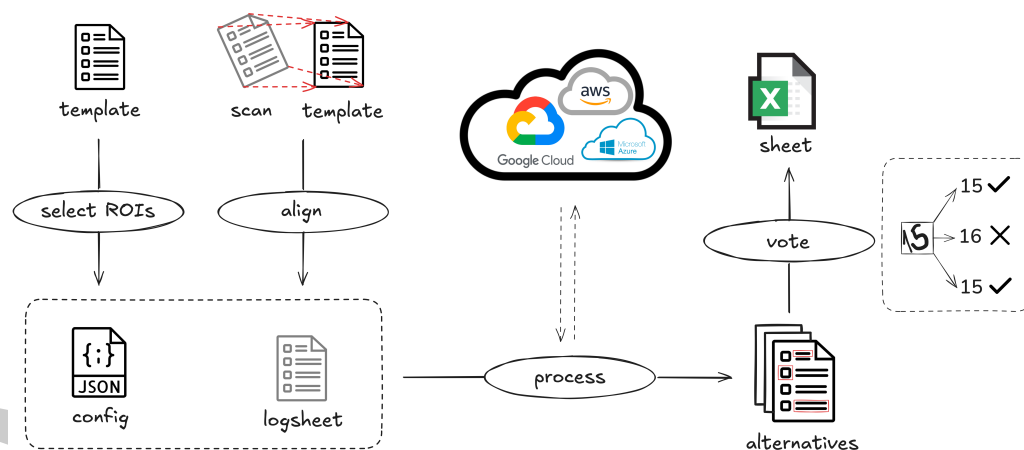


Figure 1: Schematic overview of formHTR annotation and processing workflow.

56 The next step identifies and extracts content from the ROIs. First, the scanned logsheet and
57 its template are aligned to ensure the ROIs actually match the regions in the scanned logsheet
58 (otherwise they would point to potentially empty or generally mismatched regions). While the
59 goal is straightforward, the execution can be problematic, especially when the template has no
60 fiducial markers or the scan quality is low. For this reason, a manual alignment is possible as
61 well.

62 The aligned logsheet is converted to an image and several OCR models are queried to identify
63 and extract the content. For this purpose, three services are used by calling their respective
64 APIs - Google Cloud Vision (Google LLC, 2024), Azure AI Document Intelligence (Microsoft,
65 2024), and Amazon Textract (Amazon Web Services, 2024). All three services output a set of
66 detected words with their location (bounding box) in the image.²

¹The package is not maintained anymore.

²All services offer a free tier with limited amount of requests per month. The user needs to provide suitable account credentials. A quick start description how to do this is available at the [wiki pages](#) of the formHTR repository.

67 After collecting the identified contents for each service, the process of binning assigns each
68 captured text fragment (word) to the appropriate ROI. Several cases need to be considered,
69 such as a word spanning over multiple ROIs, multiple words overlapping with a single ROI,
70 or a sentence split into several words (see examples in Figure 2). To find all overlaps for a
71 ROI from all the services, we use R-tree (Gillies, 2025) to index the regions and capture their
72 overlaps for all services' outputs and the template specification, and consequently use it to
73 effectively query for intersections. Additionally, the logsheet can contain preprinted contents
74 (*residuals*) that do not belong to ROIs, but can be shifted to its boundary box by an imperfect
75 alignment. This is handled by defining the residuals already in the specification step, thus
76 allowing to exclude them from the outputs.

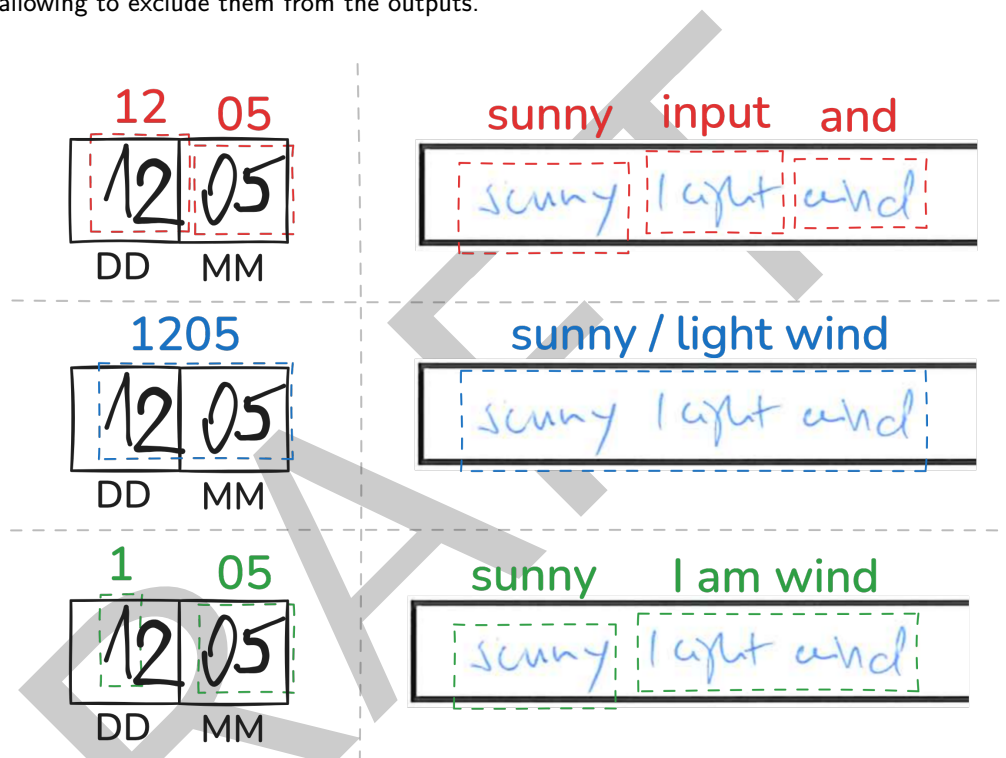


Figure 2: Examples of different scenarios of text position identification relative to a ROI. Each row (distinguished by color) corresponds to a different service. In the left column, two regions are in close proximity, causing the identified text to overlap with a neighboring region or to be recognized as a single word. In the right column, the detected words and sentence splitting into words vary across services.

77 Assuming the words are binned for all services, we can use a voting algorithm to pick the most
78 likely output. With three services, a majority vote can be applied. In cases when there is
79 no consensus, a random choice is made. Similarly, the tool works even with less than three
80 services enabled, albeit with lower output quality due to the inability to perform the voting. On
81 top of the voting, the weight of votes can be altered by using the known information about the
82 ROIs. In the current version, priority is given to numerical values (if expected). This aspect
83 has a very high potential for extensions in the future versions (e.g. allow only values from a
84 dictionary, only content satisfying a regex, only numbers from an integer range, or content
85 satisfying a length threshold).

86 The output of formHTR is an Excel spreadsheet (an .xlsx file) with two sheets. Metadata
87 sheet has three columns - a variable name coming from the specification, the extracted content
88 as the result of the voting algorithm, and a picture cut out of the scanned (and aligned)
89 logsheet based on the bounding box defined in the specification. This can be used for a quick
90 proofreading of the outputs. Extra sheet contains any miscellaneous content that neither

falls into any ROI nor was filtered out as a residual. This can typically alert the user to any comments and notes written on unexpected parts of the logsheet.

Research impact statement

The need for formHTR tool arose during the TRaversing European Coastlines (TREC) expedition³. The tool was developed and continuously optimized for the expedition, and consequently used on a collection of approximately ten thousand double-sided logsheets spanning around hundred different template types, together providing contextual metadata to more than eighty thousand collected samples. Using formHTR, the logsheet templates were annotated and all the scanned logsheets processed. The metadata was extracted and curated, and archived⁴ in the BioSamples database (Courtot et al., 2022). The contextual metadata is an essential foundation for sample analysis and data production, which, at the time of writing is in the initial phase.

Example workflow

An example workflow where we first create and annotate ROIs for the template, and consequently process the scanned logsheet, ultimately creating the output Excel spreadsheet.

```
# 1) Create ROI config for a template
formhtr select-rois --pdf-file template.pdf --output-file config.json

# 2) Annotate ROI types and variable names
formhtr annotate-rois --pdf-file template.pdf --config-file config.json --
output-file config_annotated.json

# 3) Process a scanned logsheet
formhtr process-logsheet \
  --pdf-logsheet scan.pdf \
  --pdf-template template.pdf \
  --config-file config_annotated.json \
  --output-file output.xlsx \
  --google google_credentials.json \
  --amazon amazon_credentials.json \
  --azure azure_credentials.json
```

Author's Contributions

MT wrote the manuscript and developed the software. JG contributed to the software. SP, MK, and KL contributed via conceptual guidance and contributed to the manuscript. PB provided conceptual oversight and funding.

AI usage disclosure

No generative AI tools were used in the development of this software, or the writing of this manuscript. AI tools were used in the preparation of supporting materials, namely setting up and generating the documentation and tests.

³<https://www.embl.org/about/info/trec/>

⁴<https://www.ebi.ac.uk/biosamples/samples?text=Traversing+European+Coastlines+%28TREC%29+expedition>

Acknowledgements

This publication was enabled by the support of EMBL member states to the TREC expedition (within the framework of EMBL's Molecules to Ecosystems Programme 2022-2026) and partially funded by the European Union's Horizon 2020 research and innovation program (project BIOcean5D with grant agreement No. 101059915). Views and opinions expressed are, however, those of the authors only and do not necessarily reflect those of the European Union. Neither the European Union nor the granting authority can be held responsible for them.

References

- Amazon Web Services. (2024). *Amazon textract*. <https://aws.amazon.com/textract>.
- Barlow, J. R. (2026). *OCRmyPDF* (Version 17.4.1). <https://pypi.org/project/ocrmypdf/>
- Brewer, P. W., & Guiterman, C. H. (2016). A new digital field data collection system for dendrochronology. *Dendrochronologia*, 38, 131–135. <https://doi.org/10.1016/j.dendro.2016.04.005>
- Caltech Library. (2020). *Handprint: A tool for comparing handwriting recognition services*. <https://caltechlibrary.github.io/handprint/>.
- Courtot, M., Gupta, D., Liyanage, I., Xu, F., & Burdett, T. (2022). BioSamples database: FAIRer samples metadata to accelerate research data management. *Nucleic Acids Research*, 50(D1), D1500–D1507.
- Cui, C., Sun, T., Lin, M., Gao, T., Zhang, Y., Liu, J., Wang, X., Zhang, Z., Zhou, C., Liu, H., Zhang, Y., Lv, W., Huang, K., Zhang, Y., Zhang, J., Zhang, J., Liu, Y., Yu, D., & Ma, Y. (2025). *PaddleOCR 3.0 technical report*. <https://arxiv.org/abs/2507.05595>
- Gillies, S. (2025). *Rtree: Spatial indexing for python* (Version 1.4.1). <https://pypi.org/project/rtree/>
- Google LLC. (2024). *Google cloud vision API*. <https://cloud.google.com/vision>.
- Google, & HP. (2021). *Tesseract OCR engine*. <https://github.com/tesseract-ocr/tesseract>.
- Jaided AI. (2024). *EasyOCR: End-to-end multi-lingual optical character recognition (OCR) solution* (Version 1.7.2). <https://pypi.org/project/easyocr/>
- maxentlabs. (2022). *Ocrpy: OCR, archive, index and search any documents with ease*. <https://github.com/maxent-ai/ocrpy>; GitHub.
- Microsoft. (2024). *Azure AI document intelligence (form recognizer)*. <https://learn.microsoft.com/azure/applied-ai-services/form-recognizer>.
- Mori, S., Suen, C. Y., & Yamamoto, K. (1992). Historical review of OCR research and development. *Proceedings of the IEEE*, 80(7), 1029–1058. <https://doi.org/10.1109/5.156468>
- Reul, C., Christ, D., Hartelt, A., Balbach, N., Wehner, M., Springmann, U., Wick, C., Grundig, C., Büttner, A., & Puppe, F. (2019). OCR4all—an open-source tool providing a (semi-)automatic OCR workflow for historical printings. *Applied Sciences*, 9(22). <https://doi.org/10.3390/app9224853>
- van Tamelen, P. G. (2004). A comparison of obtaining field data using electronic and written methods. *Fisheries Research*, 69(1), 123–130. <https://doi.org/10.1016/j.fishres.2004.01.006>